Distributed Multi-Model Analytics for E-commerce Data

MongoDB (document) + HBase (wide-column) + Apache Spark (distributed processing)

AUCA Big Data Analytics - Final Project - Technical Report | **DUSENGIMANA Victor** (ID 101372) | due 11 Nov 2026

Run summary. Dataset scale 0.02 of the specification: 1.000 users, 500 products, 40.000 sessions, 10.000 transactions (90 days).

1. Introduction and system architecture

The goal is to answer business questions about a synthetic e-commerce platform by using each technology where it is strongest. **MongoDB** stores entity data that is read as whole documents and aggregated along many dimensions (catalog, customers, orders). **HBase** stores the high-volume, append-only session event stream with a row key designed for time-ordered, per-user access. **Apache Spark** performs full scans, joins and algorithms (cleaning, co-occurrence, cohorts, customer lifetime value) and acts as the **integration layer** that joins MongoDB-resident data with HBase-resident data. Figure 1 shows how they fit together; section 7 explains the technology choices.

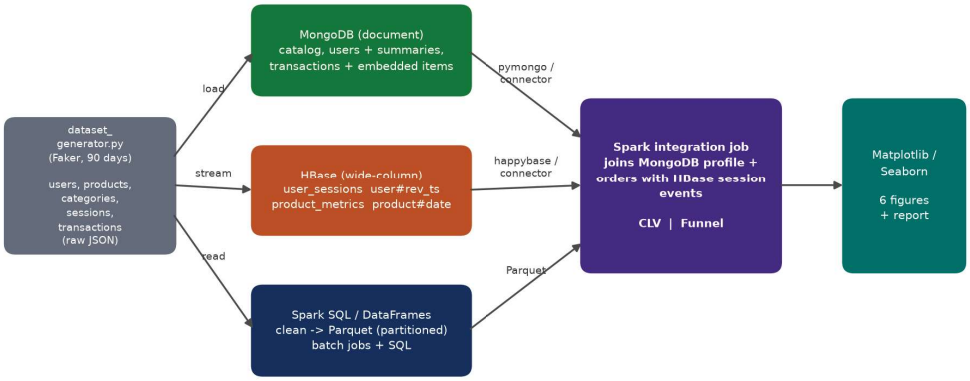


Figure 1 - System architecture: raw JSON is loaded into MongoDB and HBase and processed by Spark; the integration job joins both stores.

2. Data modelling (Part 1)

2.1 MongoDB document model

The model follows the access patterns: (i) revenue and popularity analytics over orders, (ii) customer profile and segmentation reads, (iii) product look-ups with their category. Rule of thumb used: **embed** data that is always read together and has a bounded size; **reference or denormalise** data that is shared or queried independently.

Collection	Design decision	Justification
categories	Subcategories (with profit_margin) embedded as an array	Small, 1-to-few, always read together; the whole hierarchy is one document per category.
products	price_history embedded; category and subcategory name/margin **denormalised** into the product	Price history is only read with its product. Denormalising avoids a $lookup for every product listing. *The generator does not emit subcategory_id, so one is assigned deterministically (product number mod number of subcategories).*
transactions	Line items embedded; every item also stores category_id and profit_margin; timestamp stored as a real Date (field ts)	An order is written and read atomically with its items. Revenue/profit by category needs no join. A real Date type enables $dateToString grouping and range queries on the ts index.
users	Profile + embedded, pre-computed **purchase_summary** (completed/shipped/delivered orders only, same definition as revenue) and **engagement_summary** (computed pattern)	Profile pages and segmentation hit a single document; $bucket over summary fields is a one-collection pass. Trade-off: the summaries are redundant and must be refreshed by a batch job or change stream.

Indexes (matching the pipelines): transactions(ts), (user_id, ts), (items.product_id); products(category.category_id); users(purchase_summary.total_orders), (geo_data.state). **Sessions are deliberately not stored in MongoDB**: they are large, write-heavy time-series documents (up to 16 page views each) better served by HBase and Spark.

Example transaction document (real record from the generated data, after loading transformations):

```
{
 "_id": "txn_187f1fcf4064",
 "user_id": "user_000724",
 "ts": "ISODate(2026-08-25T19:09:30)",
 "items": [
   {"product_id": "prod_00032", "quantity": 3, "unit_price": 144.47, "subtotal": 433.41,
    "category_id": "cat_008", "profit_margin": 0.36},
   {"product_id": "prod_00184", "quantity": 3, "unit_price": 166.18, "subtotal": 498.54,
    "category_id": "cat_020", "profit_margin": 0.26}
 ],
 "subtotal": 1339.45,
 "discount": 0,
 "total": 1339.45,
 "status": "completed",
}
```

Four aggregation pipelines implemented in mongodb/02_aggregations.py: **A** revenue and estimated profit by category and month ($match, $unwind, $group with $dateToString, $lookup); **B** top-20 products by revenue with category name; **C** user segmentation by purchase frequency with $bucket whose boundaries are the *quintiles of the real order-count distribution* (so it stays meaningful at any scale); **D** revenue by state and payment method ($lookup to users).

2.2 HBase wide-column model

Table / key	Column families	Why
user_sessions row key = user_id#(9999999999999 - start_epoch_ms)	**meta**: session attributes, device_*, geo_* **pv**: one sparse column per page view (pv:000, pv:001...) **cart**: cart:<product_id> = qty|price	A user's sessions are adjacent and **newest-first** (reverse timestamp), so 'latest N sessions' is one short prefix scan and a date range is one contiguous range scan. user_id leads the key, which spreads writes (no hotspot) and the table is pre-split. Families are split by access pattern: a metadata query never reads the heavy pv family. Only 3 families (each family = separate files/memstore).
product_metrics row key = product_id#yyyymmdd	**m**: views, view_seconds, cart_adds, cart_units, purchase_sessions, purchased_units	A product's daily history is contiguous for time-range scans (view-to-purchase trends). Sparse: a product-day without activity has no row.

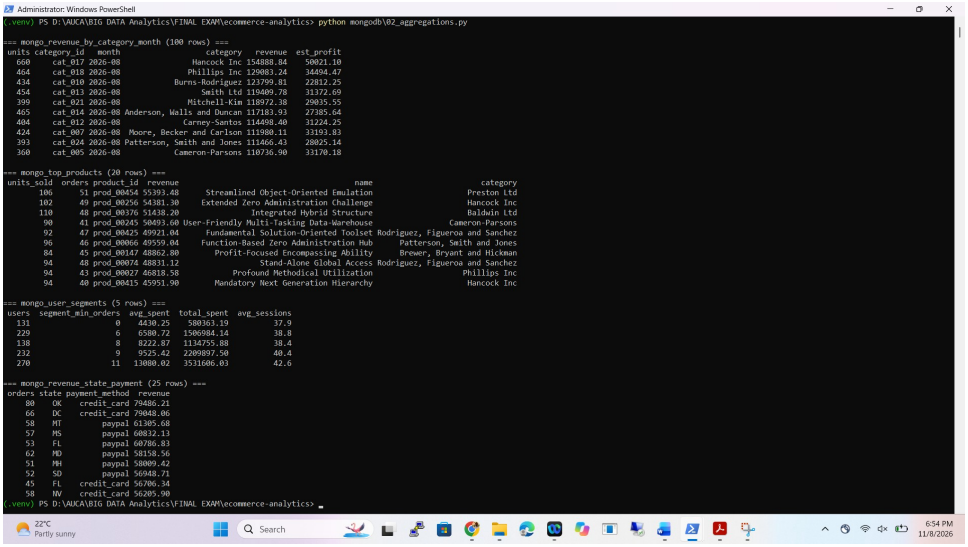
Example HBase row (real session from the generated data):

```
ROW KEY  user_000872#8212146851999
   meta:start_time = 2026-08-27T17:52:28
   meta:duration_seconds = 2017
   meta:device_type = desktop
   meta:conversion_status = abandoned
   pv:000 = 2026-08-27T17:52:28|home|||34
   cart:prod_00154 = 2|594.15
```

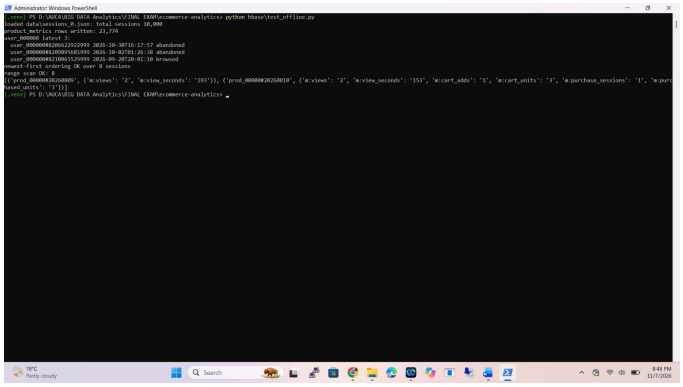
2.3 Why some data is in MongoDB and some in HBase

Need	Best fit	Reason
Ad-hoc, multi-dimensional aggregation over orders and customers	MongoDB	Aggregation framework, secondary indexes, nested documents; no sorted-key constraint.
Fetch one user's recent activity in milliseconds; time-range scans over events	HBase	Sorted row keys, prefix/range scans, wide sparse rows, very high write throughput, linear scale-out.
Heavy joins, iterative algorithms, full scans of everything	Spark	Distributed in-memory processing; SQL and DataFrames; reads any of the stores.

HBase has no secondary indexes or joins, so any question that is not answered by the row key goes to Spark; MongoDB is not designed for billions of tiny events. This is why order data lives in MongoDB while click-stream data lives in HBase.



Screenshot 1 - MongoDB (real server): aggregation pipelines A-D, including A (revenue by category and month).



Screenshot 2 - HBase: loaded subset, newest-first scan of one user's sessions, range scan and product_metrics row. Offline test harness.

3. Spark processing pipeline (Part 2)

Pipeline. Raw JSON (one JSON array per file, so read with multiLine=true) -> explicit schemas (no inference pass over gigabytes) -> cleaning/normalisation job -> partitioned Parquet (sessions by date, transactions by month) -> batch analytics and Spark SQL.

Cleaning and normalisation (06_clean_normalize.py). Timestamps parsed from ISO strings; duplicates removed on primary keys; text standardised (state upper-case, device and referrer lower-case); invalid rows dropped (negative durations, non-positive prices); the category hierarchy exploded to a flat dimension; transaction totals recomputed and compared with stored values; referential-integrity checks via left-anti joins. Result of the data-quality report:

Check	Result	Check	Result
users raw/clean	(1000, 1000)	txn with unknown user	0
products raw/clean	(500, 500)	txn session_id missing in sessions	0
sessions raw/clean	(40000, 40000)	txn items with unknown product	0
transactions raw/clean	(10000, 10000)	txn dated BEFORE the user's registration	0
txn items mismatch	0	sessions 'converted'	8219
txn total mismatch	0	...converted but NO transaction (generator txn cap)	3139
txn not linked to a session	4920		

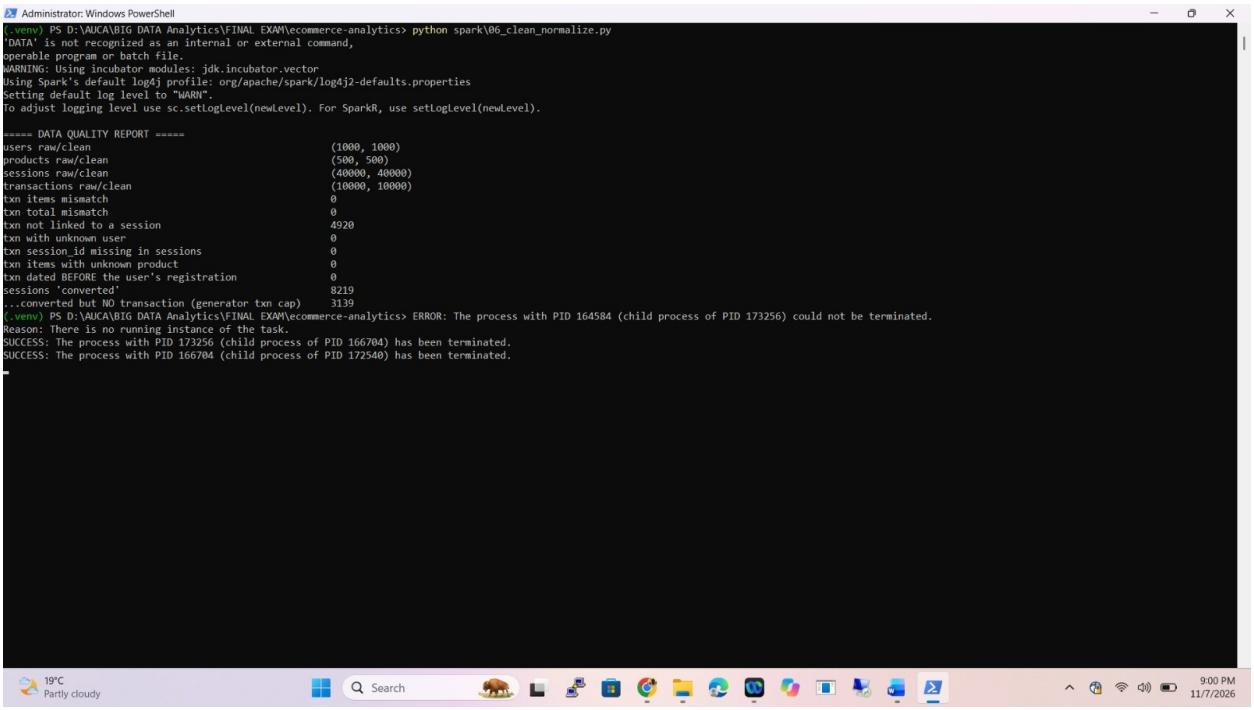
Findings. All stored totals are consistent (0 mismatches), all references resolve, and no transaction predates its user's registration. Two characteristics of the synthetic data matter: (1) 4,920 of 10,000 transactions (49%) have no session link by design (the generator creates 'extra' orders); (2) 3,139 of 8,219 sessions flagged 'converted' (38%) have *no* transaction. We traced this to the generator: once the transaction cap is reached it stops creating transactions while sessions keep being flagged converted. For funnel reconciliation we therefore use the first part of the sessions (HBase subset) where the two systems agree.

Batch analytics (07_batch_analytics.py). (a) *Frequently bought together*: self-join of basket items with a.id < b.id (each unordered pair once), with support, confidence and lift; (b) *frequently viewed together* from sessions; (c) *cohort analysis* grouping users by registration month and measuring revenue per user in each purchase month. Top pair by baskets: prod_00037 + prod_00205 (4 baskets, lift 11.9). Lift values are noisy because pair counts are small.

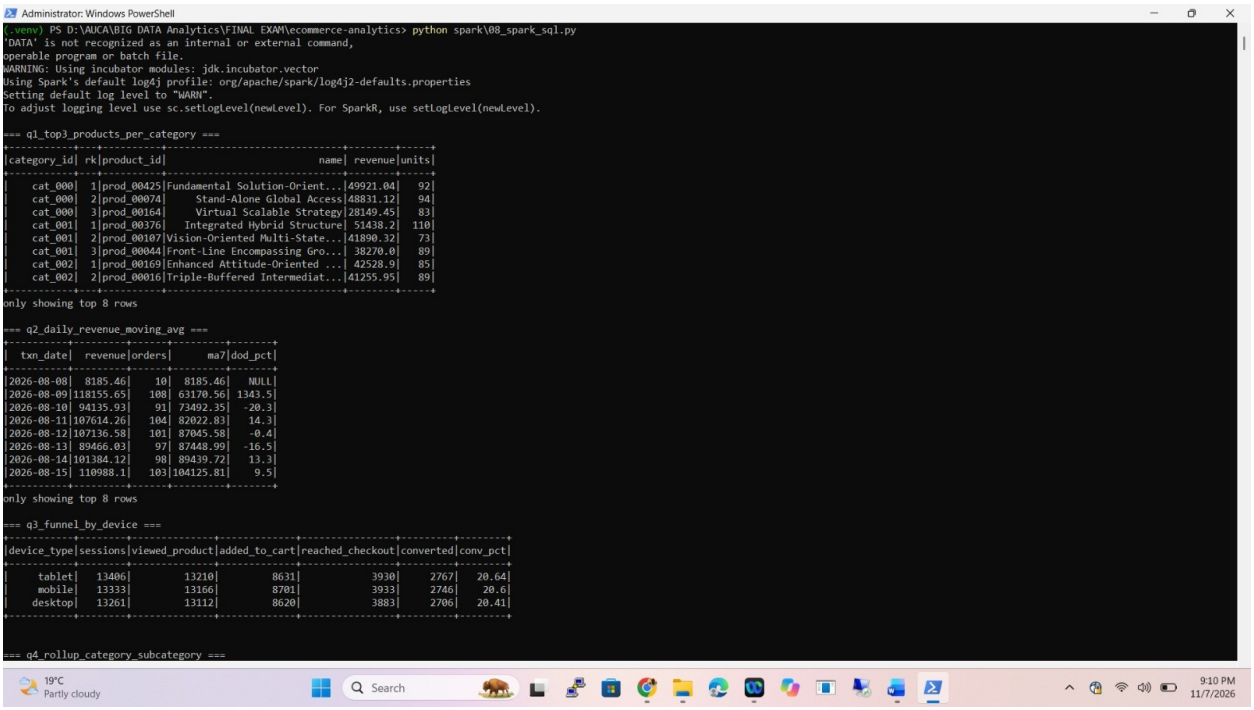
Optimisations used. Parquet (columnar) with partition pruning; column pruning; **broadcast join** of the small product dimension; pair de-duplication in the self-join; cache() for DataFrames reused several times; shuffle partitions tuned to the data size (SHUFFLE_PARTS); explicit schemas instead of inference.

Spark SQL (08_spark_sql.py). Six queries: top-3 products per category (CTE + ROW_NUMBER window), 7-day moving average and day-over-day growth (window frames, LAG), device funnel (higher-order function exists on the nested page_views array), revenue/profit ROLLUP by category and subcategory, and two queries on views (mongo_users, mongo_purchase_summary, hbase_sessions) that stand in for data loaded through the MongoDB and HBase connectors - the SQL logic is identical either way. Query 1:

```
q1_top3_products_per_category
WITH lines AS (
  SELECT t.transaction_id, i.product_id, i.quantity, i.subtotal
  FROM transactions t LATERAL VIEW explode(t.items) x AS i
  WHERE t.status IN ('completed','shipped','delivered')
), prod_rev AS (
  SELECT p.category_id, p.product_id, p.name, SUM(l.subtotal) AS revenue, SUM(l.quantity) AS units
  FROM lines l JOIN products p ON p.product_id = l.product_id
  GROUP BY p.category_id, p.product_id, p.name
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY revenue DESC) AS rk
  FROM prod_rev
)
SELECT category_id, rk, product_id, name, ROUND(revenue,2) AS revenue, units
FROM ranked WHERE rk <= 3 ORDER BY category_id, rk;
```



Screenshot 3 - Spark cleaning job and data-quality report.



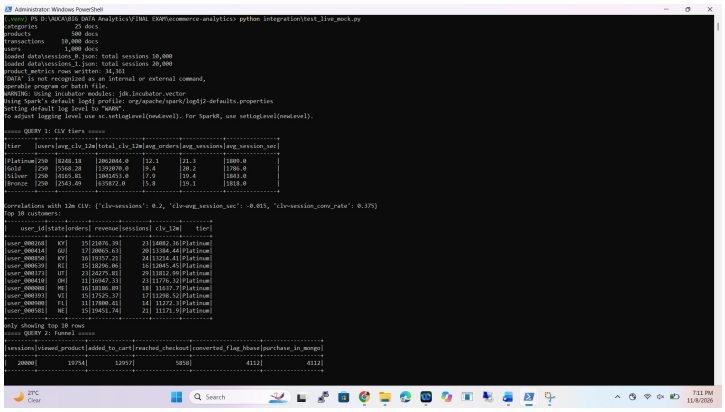
Screenshot 4 - Spark SQL queries 1 (top products per category), 2 (moving average) and 3 (device funnel).

4. Integrated analytics (Part 3)

Query 1 - Customer Lifetime Value (CLV). *Business question:* which customers will be most valuable over the next 12 months, and does browsing engagement predict value? *Sources:* MongoDB (profiles, plus a per-user revenue/profit roll-up computed **inside** MongoDB so only one small row per user leaves the database), HBase (session count, average duration and conversion per user), Spark (join, CLV, NTILE tiers, correlations). *Formula:* monthly net profit = (gross item profit - discounts) / 3 observed months; CLV_{12m} = monthly net profit x $(1 + r + ... + r^{11})$, with an **assumed** monthly retention r = 0.90 (an assumption, not estimated from data - a 90-day window cannot reveal churn).

Query 2 - Funnel conversion. *Business question:* where do shoppers drop out between first product view and a confirmed purchase, and does it differ by device or referrer? *Sources:* HBase sessions (view -> cart -> checkout, device, referrer) and MongoDB transactions (system of record for 'order exists'), joined in Spark on session_id. *Method:* the funnel is **strictly nested** - a session counts at a stage only if it also passed every earlier stage (view, cart, checkout, converted, order exists). This matters because the generator's page logs are noisy: for example, some sessions with an empty cart still contain a 'checkout' page, so counting each stage independently would overstate the later stages.

Workflow. (1) pymongo runs two aggregations in MongoDB; (2) a happybase column-restricted scan reads the HBase sessions and derives stage flags; (3) both become Spark DataFrames; (4) Spark joins on user_id / session_id, computes CLV, NTILE tiers, correlations and funnel counts; (5) results are exported to CSV for plotting. **Performance notes:** aggregation is pushed down to MongoDB; HBase scans are restricted to needed families; at scale the single-threaded Python readers are replaced by the MongoDB Spark Connector and the Spark-HBase connector so reads are parallel across executors. The same code can run from Parquet files (--source files); we verified that live and file modes produce identical results. **Verification.** tools/verify_results.py re-computes the headline numbers (revenue totals, top product, funnel stages, CLV of sample customers, the cohort table, the top product pair) from the raw JSON in plain Python and compares them with the MongoDB, Spark and integration outputs; all checks must pass (see Appendix A7).



Screenshot 5 - Integrated query 1: CLV tiers, correlations and top customers. Offline test harness for the two stores.

5. Visualisations and business insights (Part 4)

Overall (completed/shipped/delivered orders): revenue $9,196,373, estimated profit $2,378,104 (25.9% margin). Highest-revenue category: Hancock Inc ($540,051, 5.9% of revenue); best product: Streamlined Object-Oriented Emulation ($55,393, 106 units). Best day: 18 Sep 2026 ($129,397).

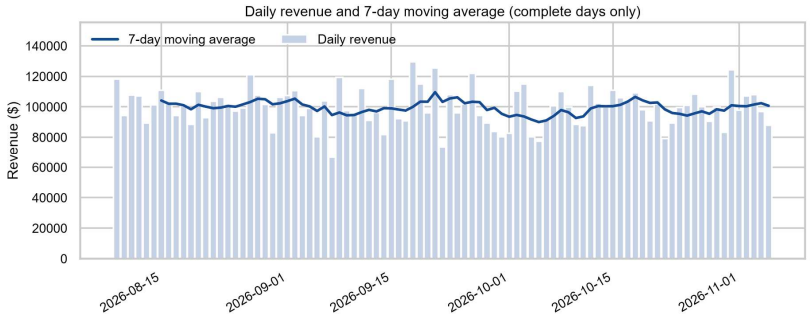


Figure 2 - Daily revenue (Spark SQL) with 7-day moving average; the partial first and last day of the window are excluded.

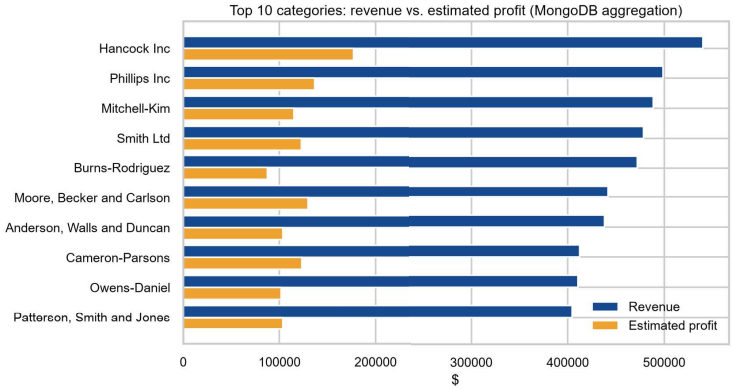


Figure 3 - Top 10 categories by revenue vs estimated profit (MongoDB aggregation A). Profit rank differs from revenue rank because margins differ by subcategory.

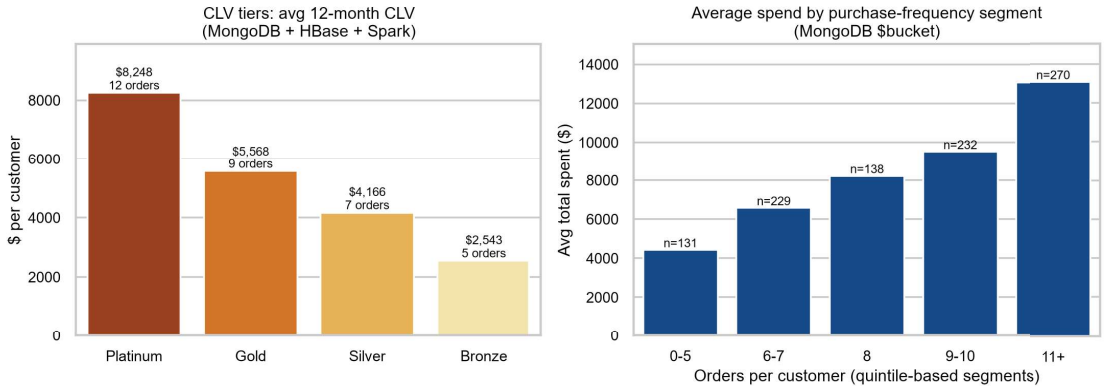


Figure 4 - Customer segmentation: CLV tiers (left) and purchase-frequency quintile segments (right).

Segmentation insight. Platinum customers (top 25%) have an average 12-month CLV of $8,248 versus $2,543 for Bronze (3.2x) and generate 40% of total projected CLV. CLV correlates with the session conversion rate (r = 0.38) and session count (r = 0.20) but hardly with time on site (r = -0.01). **Action:** invest in what turns visits into orders (offers, checkout friction) rather than in raw time-on-site; treat the Platinum tier with retention programmes.

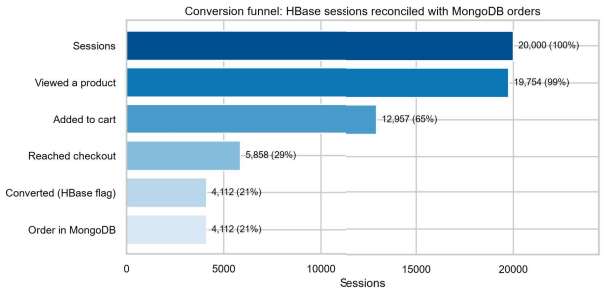


Figure 5 - Conversion funnel (HBase sessions reconciled with MongoDB orders).

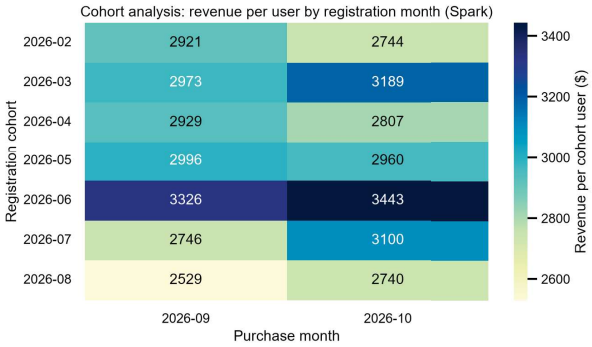


Figure 6 - Cohort analysis: revenue per user by registration and purchase month (Spark).

Funnel insight. 99% of sessions view a product, 65% add to cart, 29% reach checkout and 21% convert. The largest loss is **added to cart -> reached checkout** (55% drop). **Action:** prioritise fixes at that step (e.g. persistent carts, reminder e-mails for carts that reached checkout).

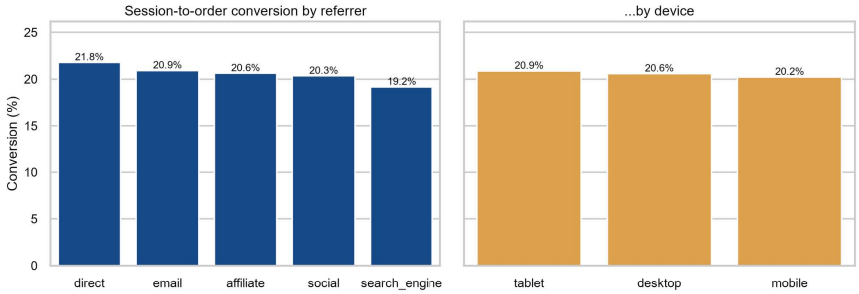


Figure 7 - Session-to-order conversion by referrer and by device.

Channel insight. Conversion by referrer ranges from 19.2% (search_engine) to 21.8% (direct), a spread of 2.6 percentage points; by device the spread is only 0.7 points (negligible). **Action:** review landing pages and bids for the weakest channel (search_engine) before changing device-specific experiences.

Cohort insight. Revenue per user varies little between cohorts and months (coefficient of variation 0.08): there is no visible decay or growth with customer age. The data are synthetic (purchases are assigned at random), so cohort patterns must not be read as real-world findings. Calendar months with fewer than 25 days of data (partial months at the start or end of the 90-day window) are excluded.

6. Scalability considerations

- **MongoDB:** shard transactions on a hashed user_id (or a time bucket) so writes spread; keep user summaries to avoid cross-shard $lookup; use compound indexes that match the pipelines; the full dataset (500k orders) fits one replica set easily.
- **HBase:** scales by region splits across RegionServers; the key leads with user_id (not the timestamp) to avoid hotspots; pre-split tables; TTL and compaction retire old sessions; batch Puts reduce RPC overhead; at 2M sessions the loader should use bulk load (ImportTsv/HFiles) or the Spark-HBase connector.
- **Spark:** partitioned Parquet (the 4 GB of JSON shrinks several-fold), broadcast of small dimensions, tuned shuffle partitions, explicit schemas; the same code runs on YARN/Kubernetes by changing the master. Avoid collecting large results to the driver.

7. Technology selection summary

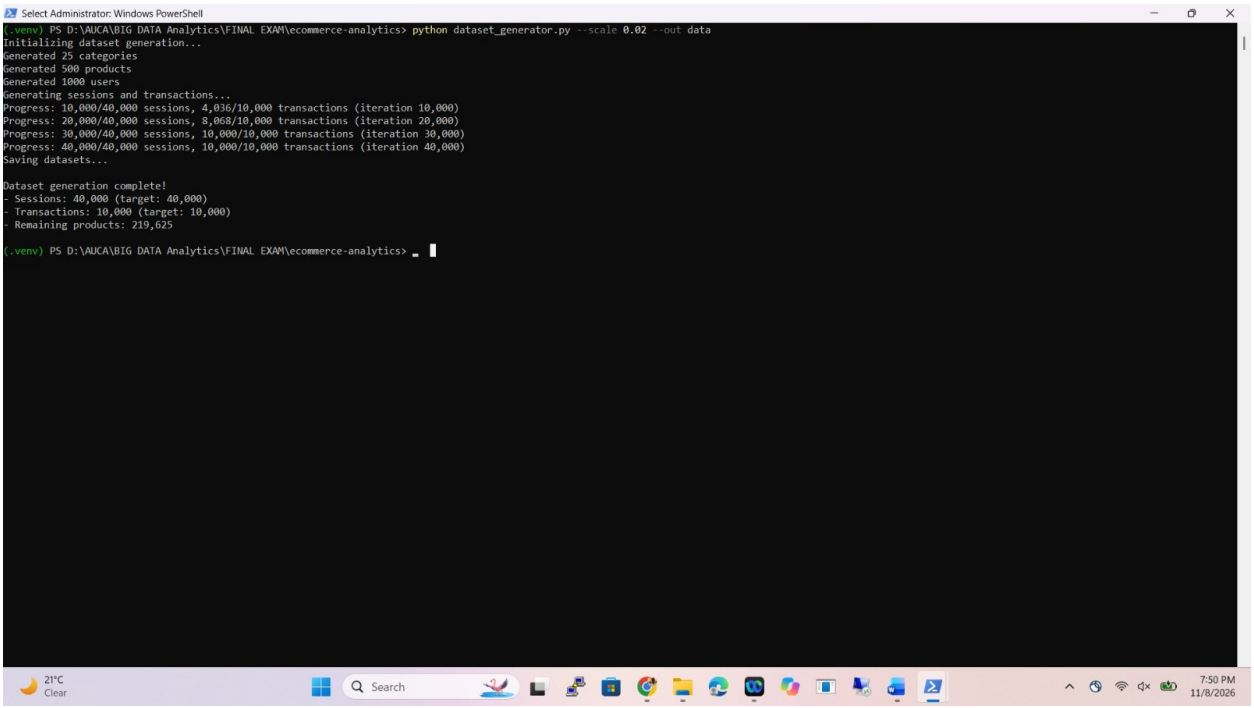
Task	Technology	Why
Revenue, profit, top products, customer summaries	MongoDB	Flexible nested documents + aggregation pipeline + indexes.
User session timeline, product daily metrics	HBase	Time-ordered row keys, range scans, sparse wide rows.
Cleaning, recommendations, cohorts, SQL analytics	Spark	Distributed scans and joins over all sessions; SQL + DataFrames.
CLV and funnel	All three	Needs orders (Mongo) + engagement (HBase) + a join/compute engine (Spark).

8. Limitations and future work

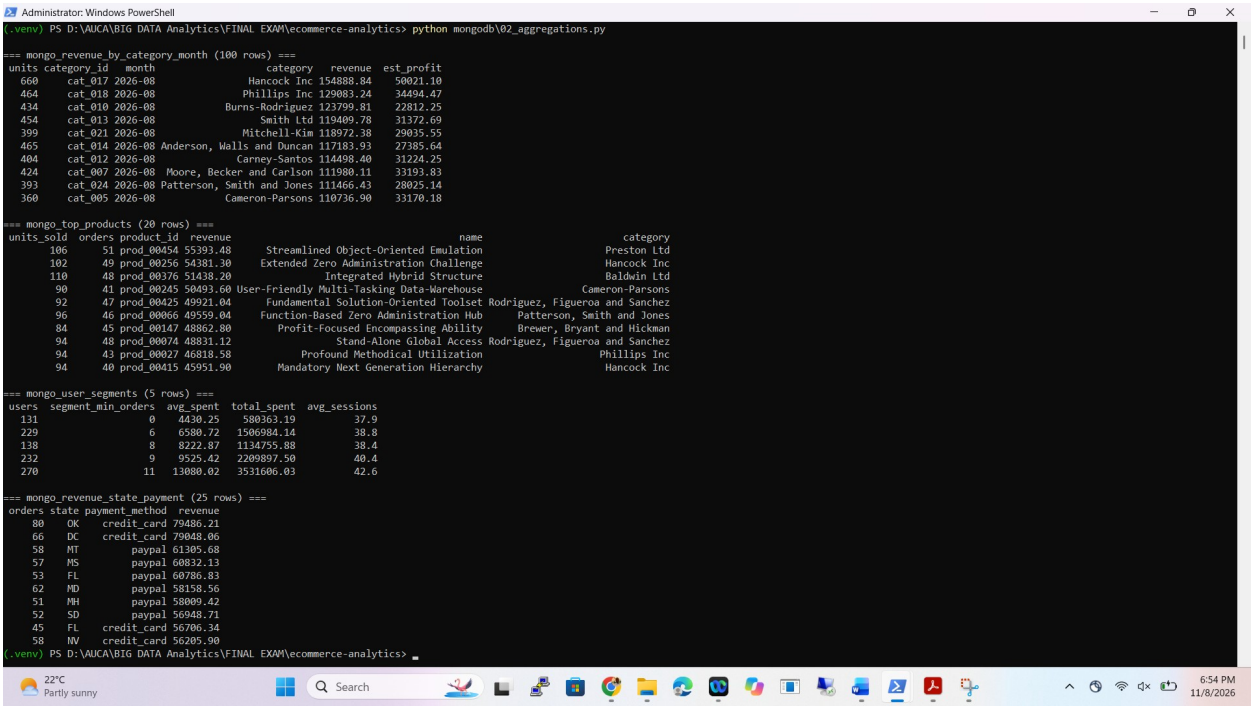
- **Synthetic data:** purchases and sessions are random, so cohorts, segments and correlations are weak or flat; lift on small baskets is noisy. Findings demonstrate the method, not real customer behaviour.
- **Generator artefacts:** no subcategory_id on products (assigned by rule); no age (age-group segmentation impossible); orphan transactions; converted sessions without orders after the transaction cap.
- **CLV** uses a fixed assumed retention (0.90) and a 90-day window; there is no churn model. MongoDB holds all orders and users, but HBase was loaded with a subset of the sessions (first session files); the integration reads through single-threaded Python clients, so engagement figures come from that subset.
- **Future work:** ALS collaborative filtering with Spark MLlib; Structured Streaming simulation of live sessions into HBase; MongoDB change streams to keep user summaries fresh; RFM + K-means segmentation; Kafka ingestion; connector-based parallel reads.

Appendix A - Console captures of the pipeline

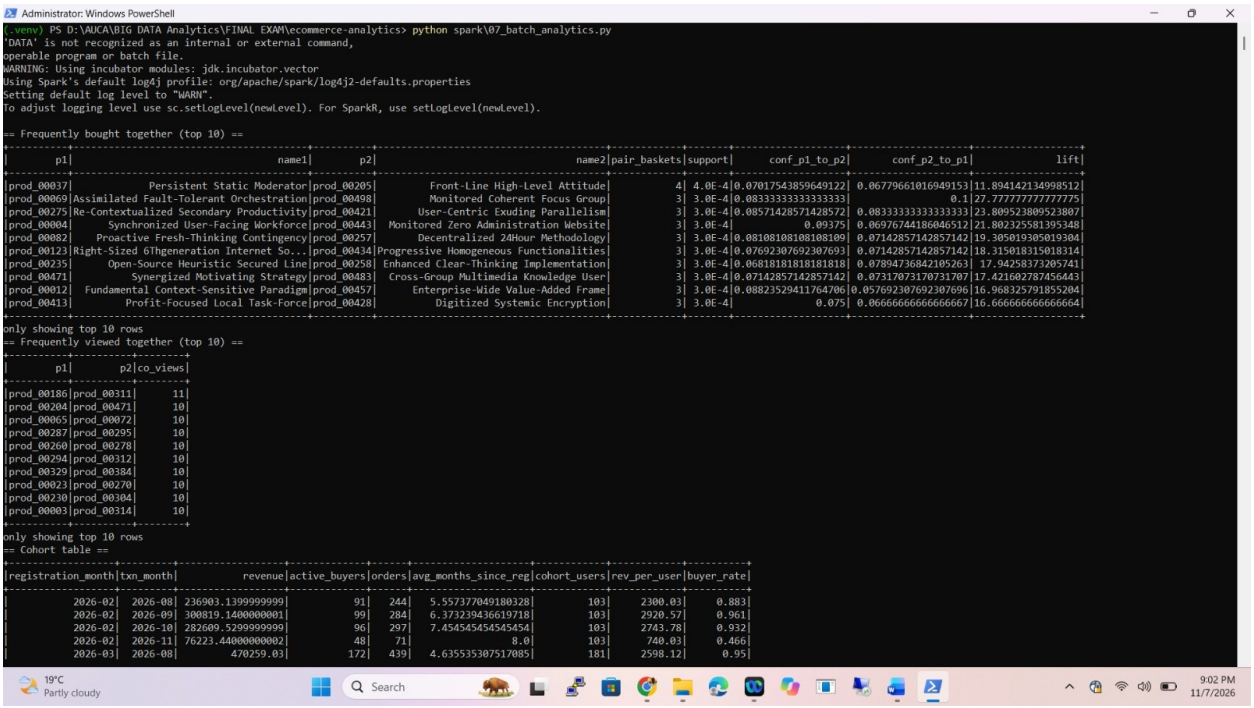
These are real screenshots of my own run of each pipeline step on Windows (PowerShell, Python 3.12, Java 17, PySpark 4.3 in local mode). MongoDB ran as a real server on the same machine (localhost:27017). HBase could not be installed without Docker, so the HBase step and the integration job used the project's offline test harness (in-memory stand-ins) for the two stores; all Spark steps and the verification ran for real.



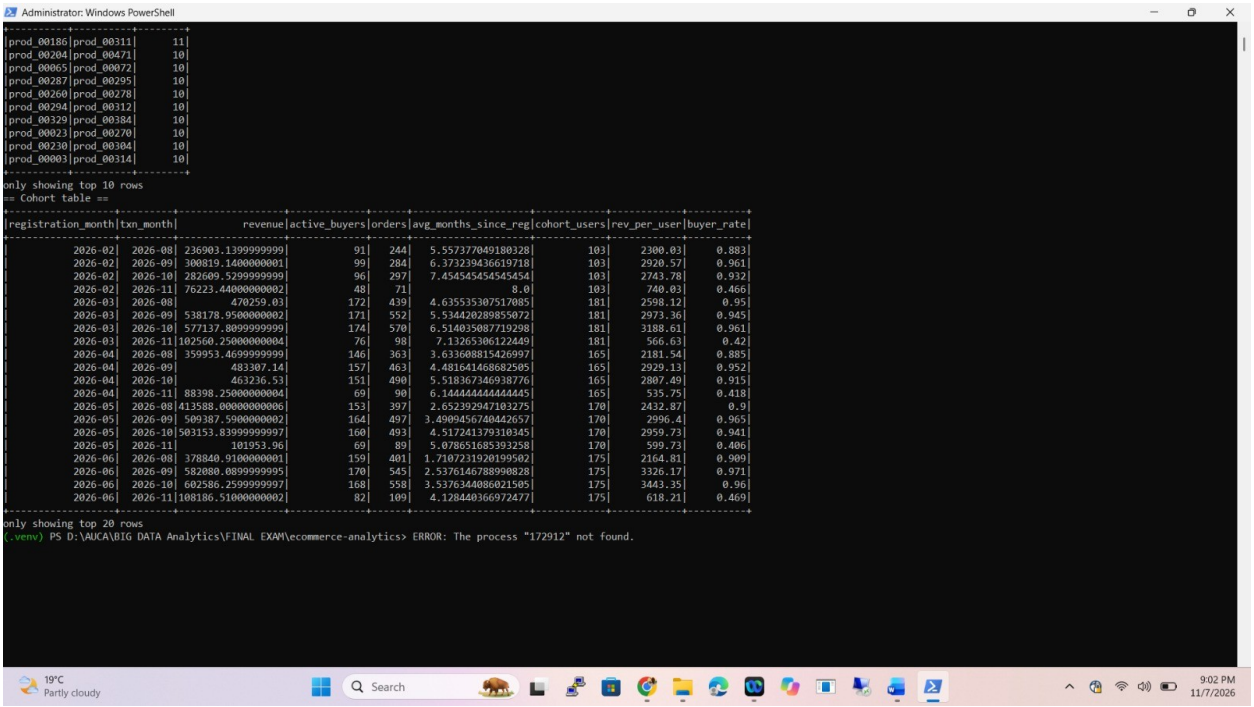
A1 - Dataset generation (dataset_generator.py --scale 0.02): 40,000 sessions and 10,000 transactions.



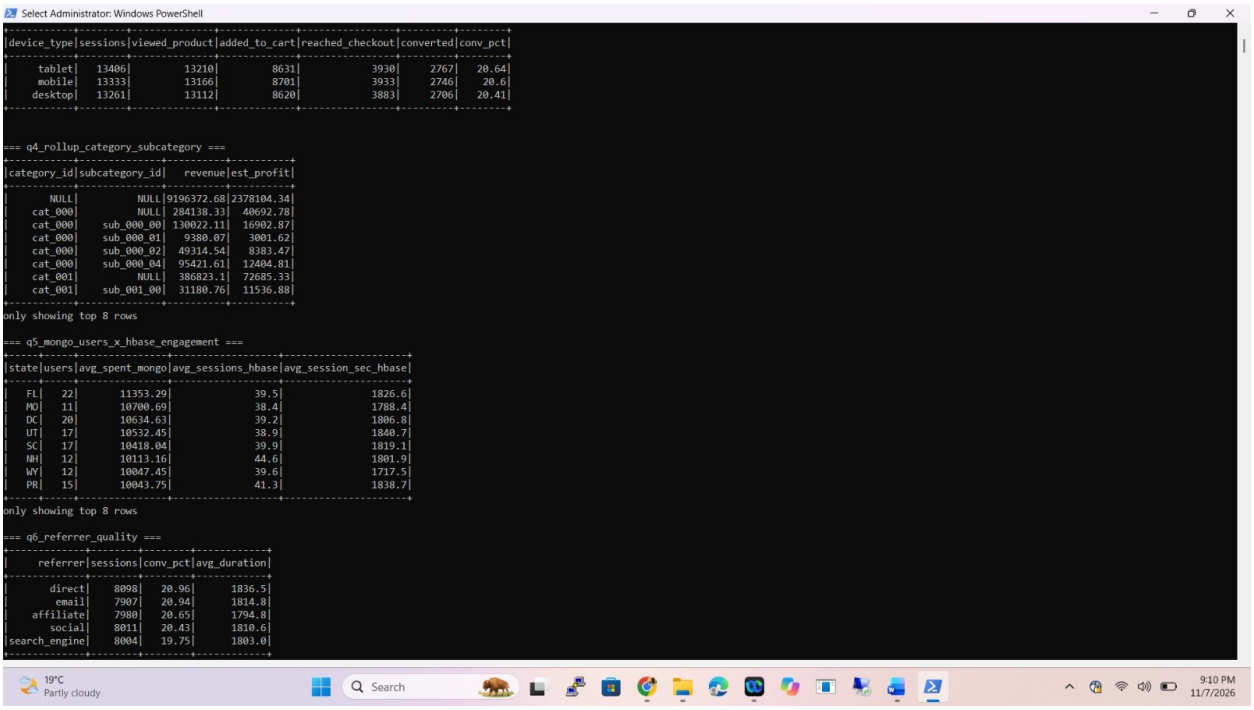
A2 - MongoDB pipelines A-D on the real MongoDB server, including B (top products) and C (segmentation by quintiles).



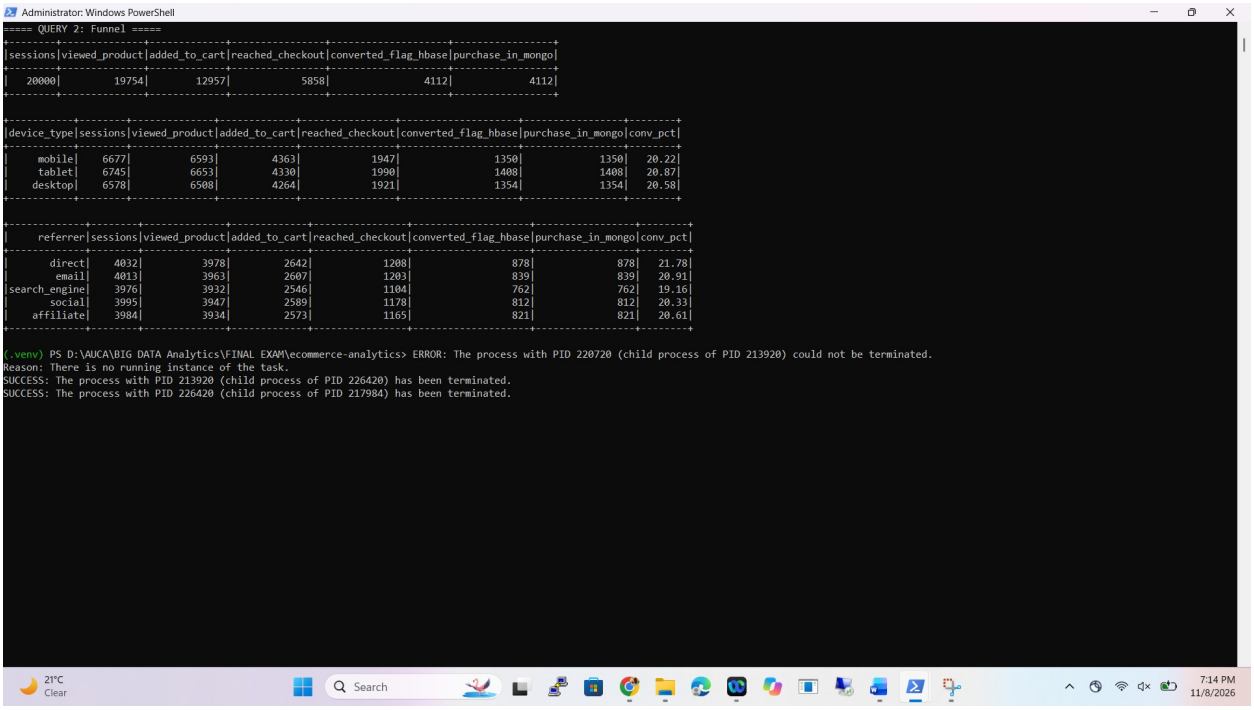
A3 - Spark: frequently bought together (support, confidence, lift) and frequently viewed together.



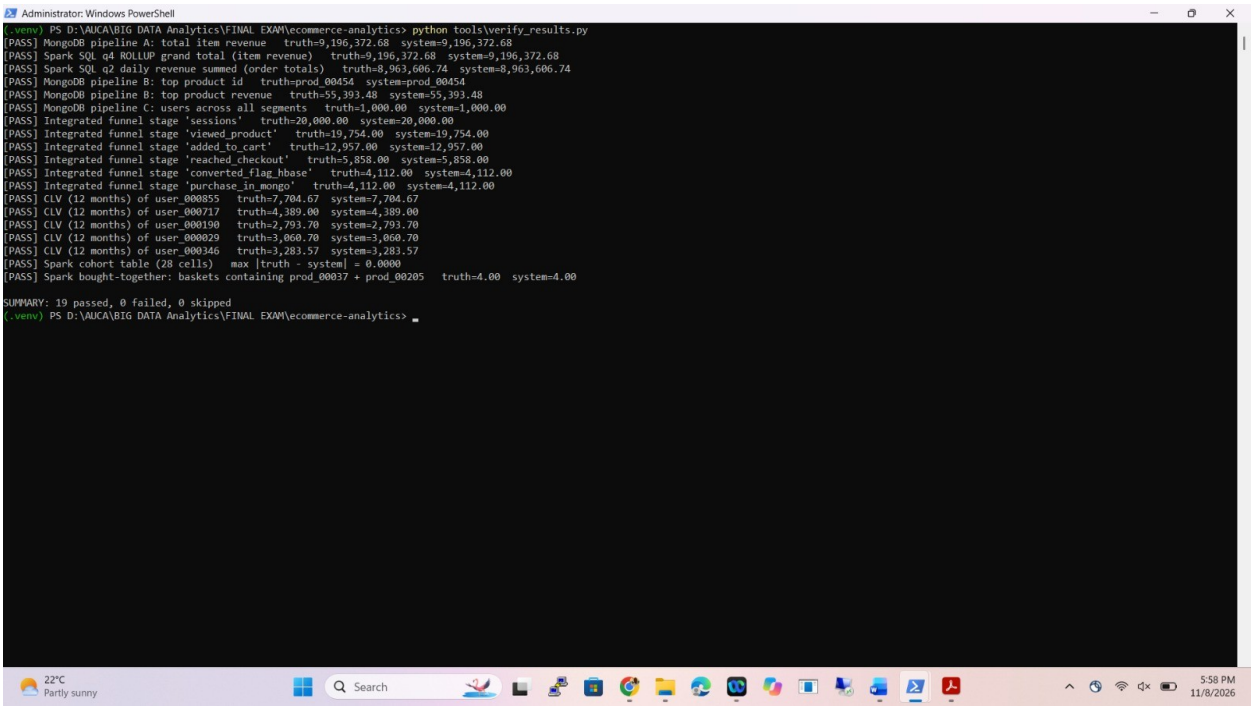
A4 - Spark: cohort table (revenue per user by registration month and purchase month).



A5 - Spark SQL: device funnel (q3), ROLLUP (q4), MongoDB x HBase engagement join (q5) and referrer quality (q6).



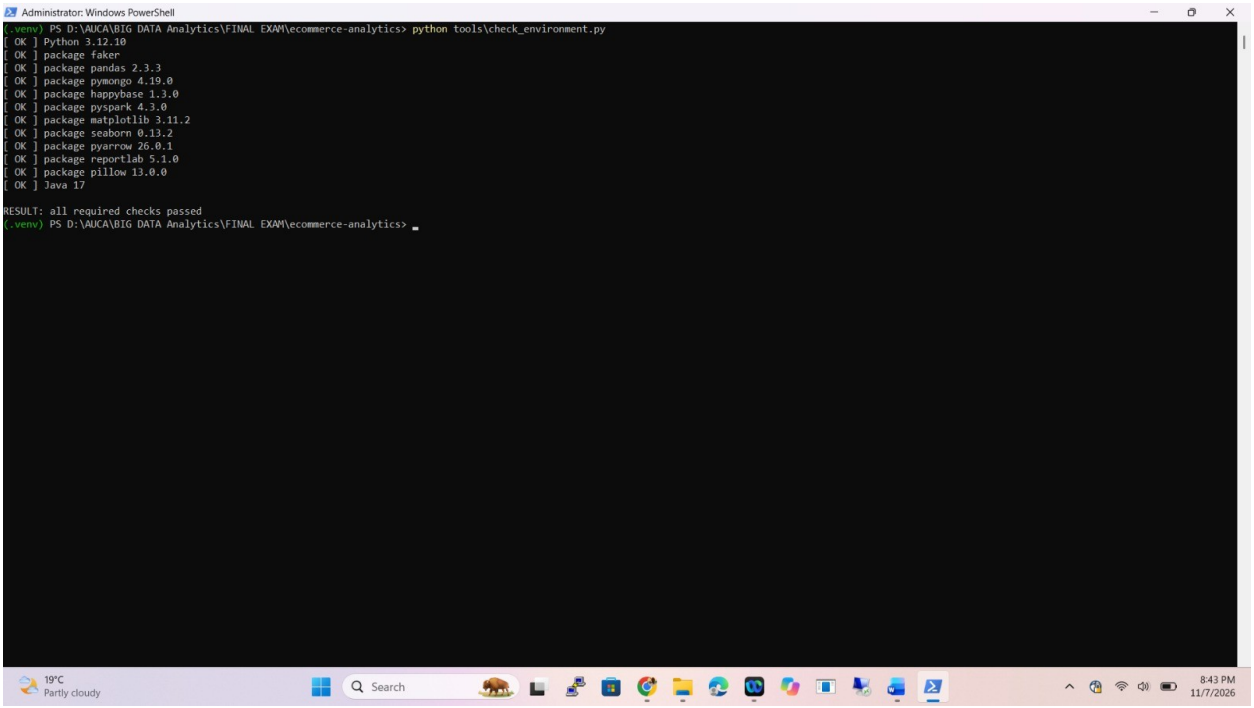
A6 - Integrated query 2: funnel (HBase sessions x MongoDB orders), overall, by device and by referrer. Offline test harness for the two stores.



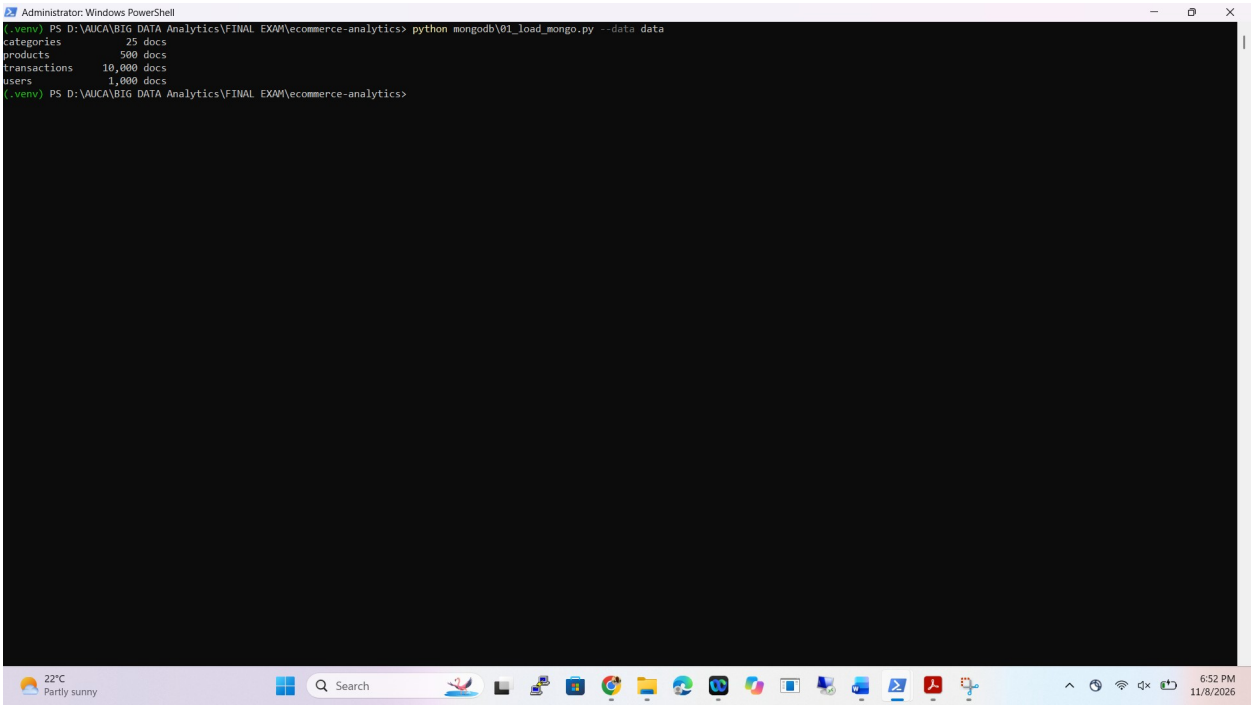
A7 - Independent verification of the results against the raw JSON (plain Python, no database or Spark): 19 passed, 0 failed.

Appendix B - Further screenshots from my own run

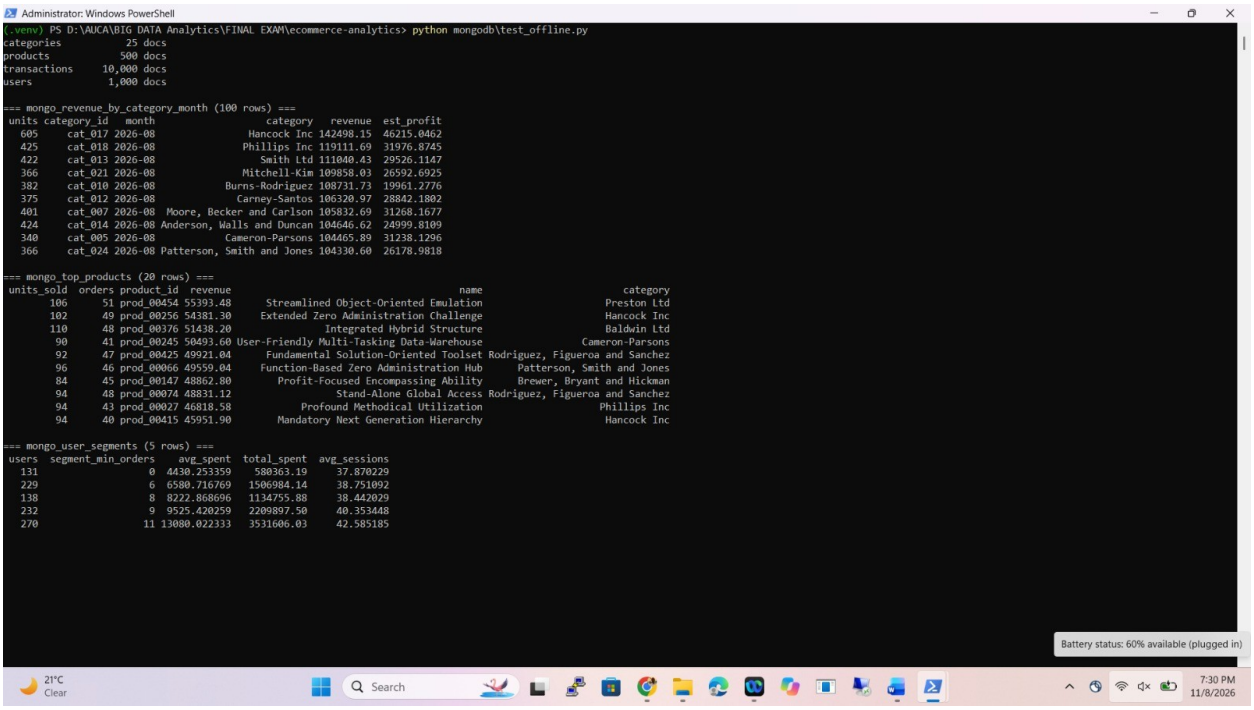
Additional real screenshots: environment check, loading the real MongoDB server, the MongoDB and HBase offline tests, the Spark cleaning job and SQL queries 1-3, integrated query 1 (CLV), the visualisation step and the Spark web UI during a Spark session.



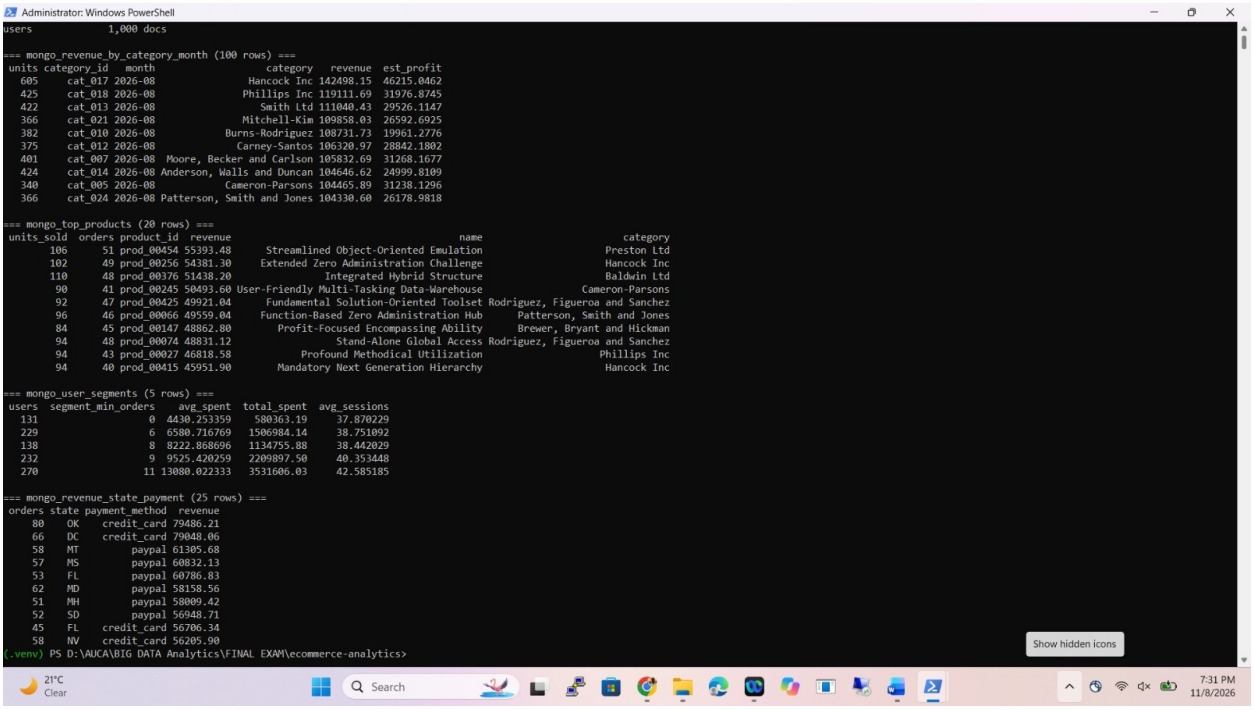
B1 - Environment check: Python 3.12, all packages and Java 17 installed.



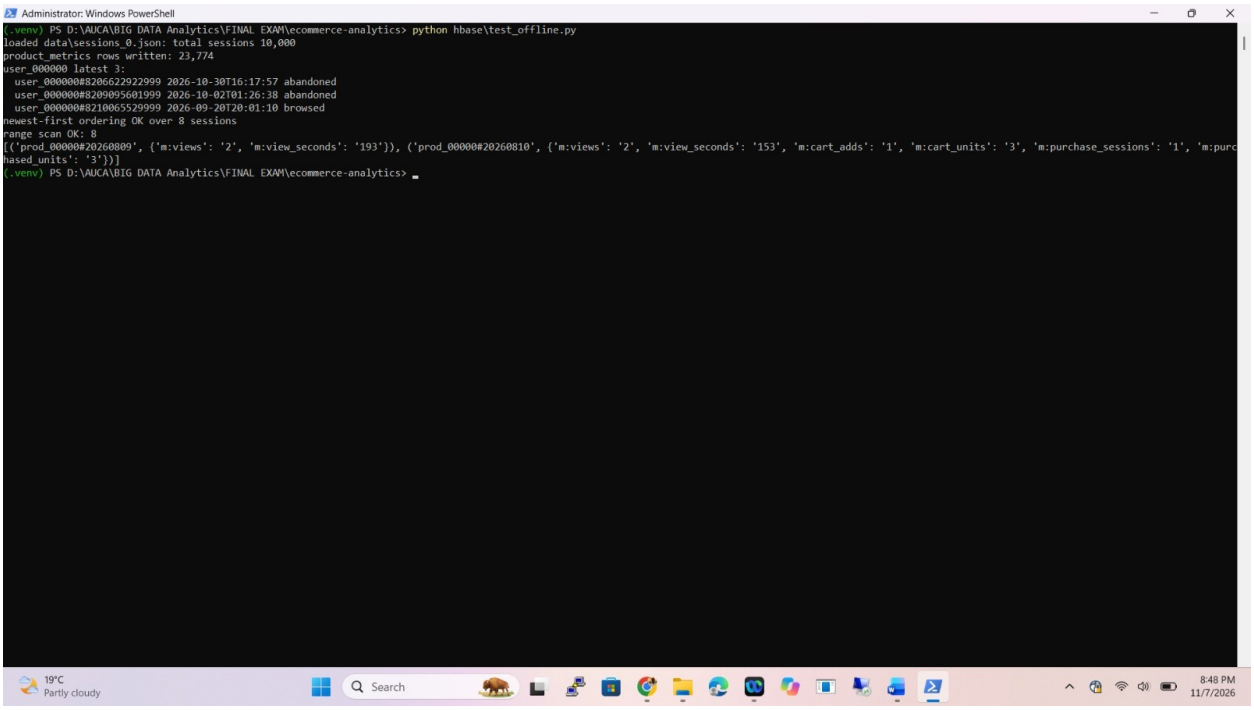
B2 - MongoDB (real server on localhost:27017): data loaded with mongodb/01_load_mongo.py - 25 categories, 500 products, 10,000 transactions, 1,000 users.



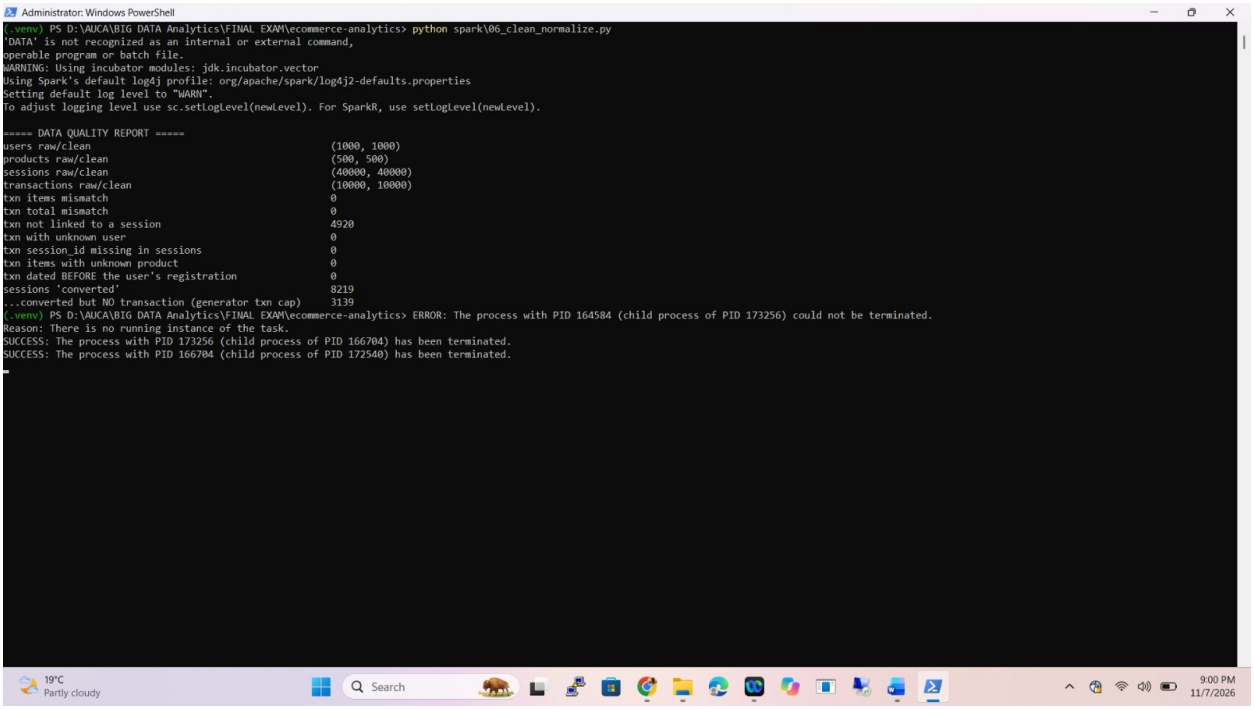
B3 - MongoDB offline test (mongodb/test_offline.py): pipeline A (revenue by category and month), B (top products) and C (segmentation). Offline test harness.



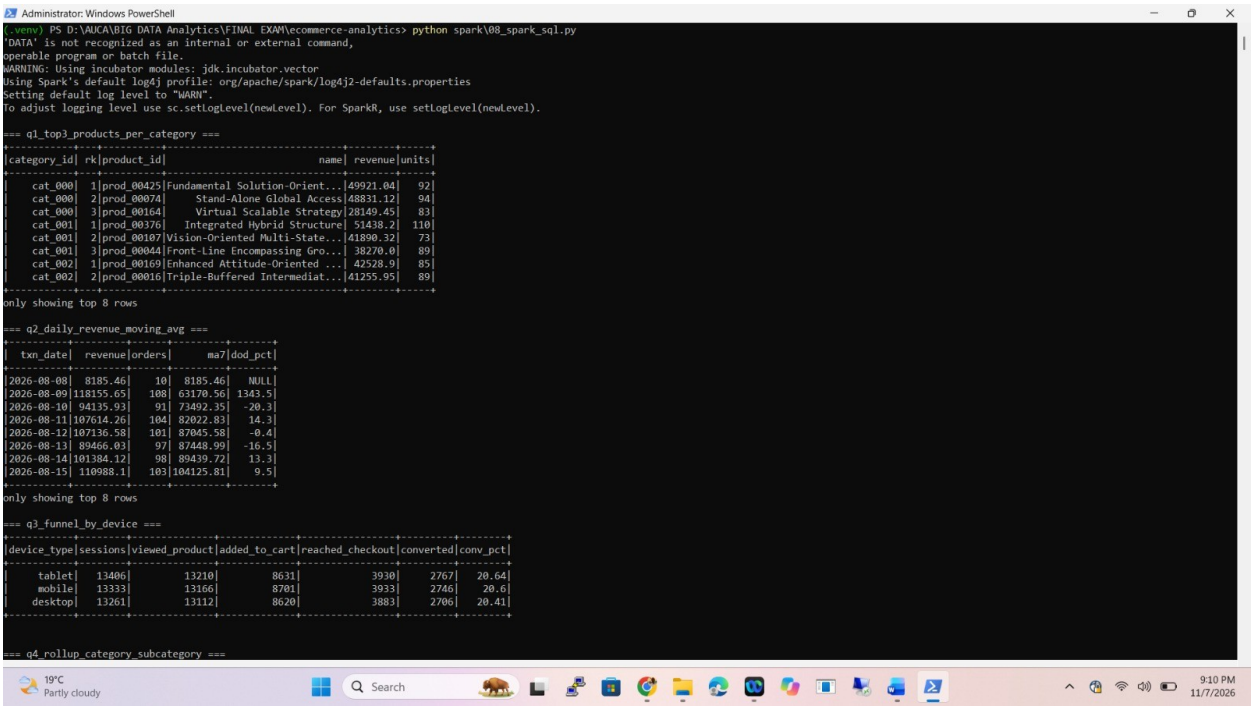
B4 - MongoDB offline test, continued: pipelines B, C and D (revenue by state and payment method). Offline test harness.



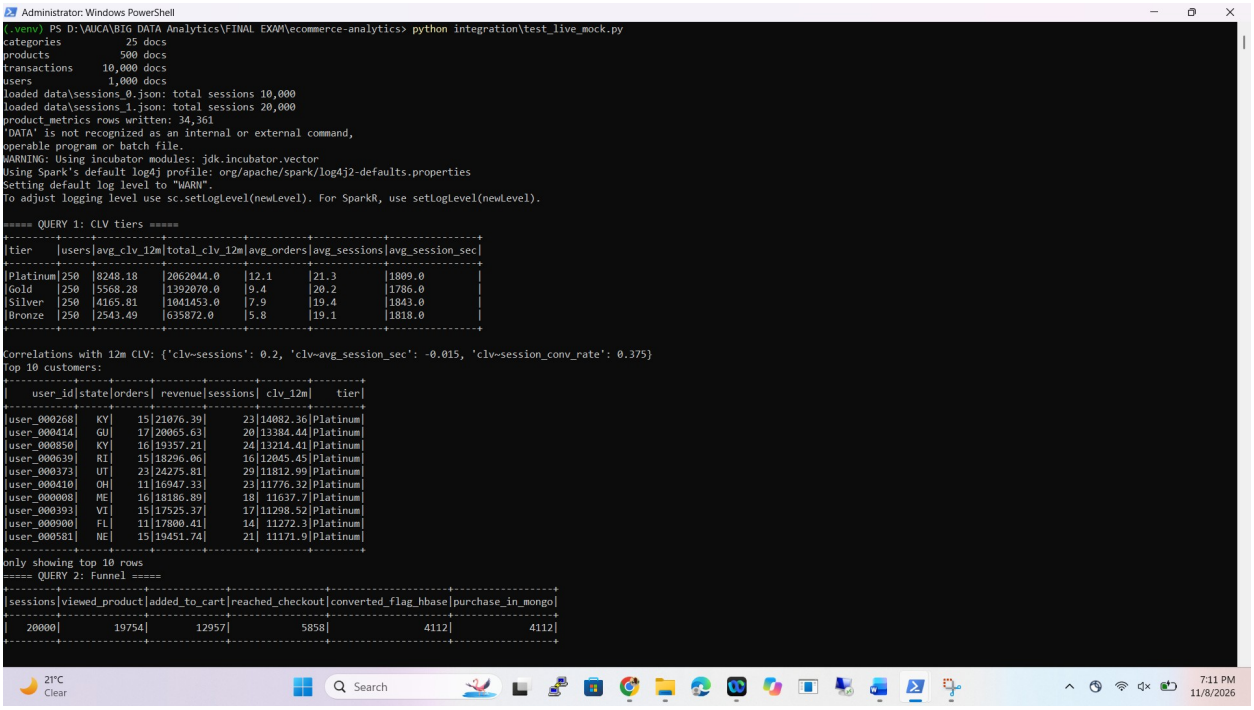
B5 - HBase: sessions loaded, newest-first scan of one user's sessions, range scan and product_metrics rows. Offline test harness.



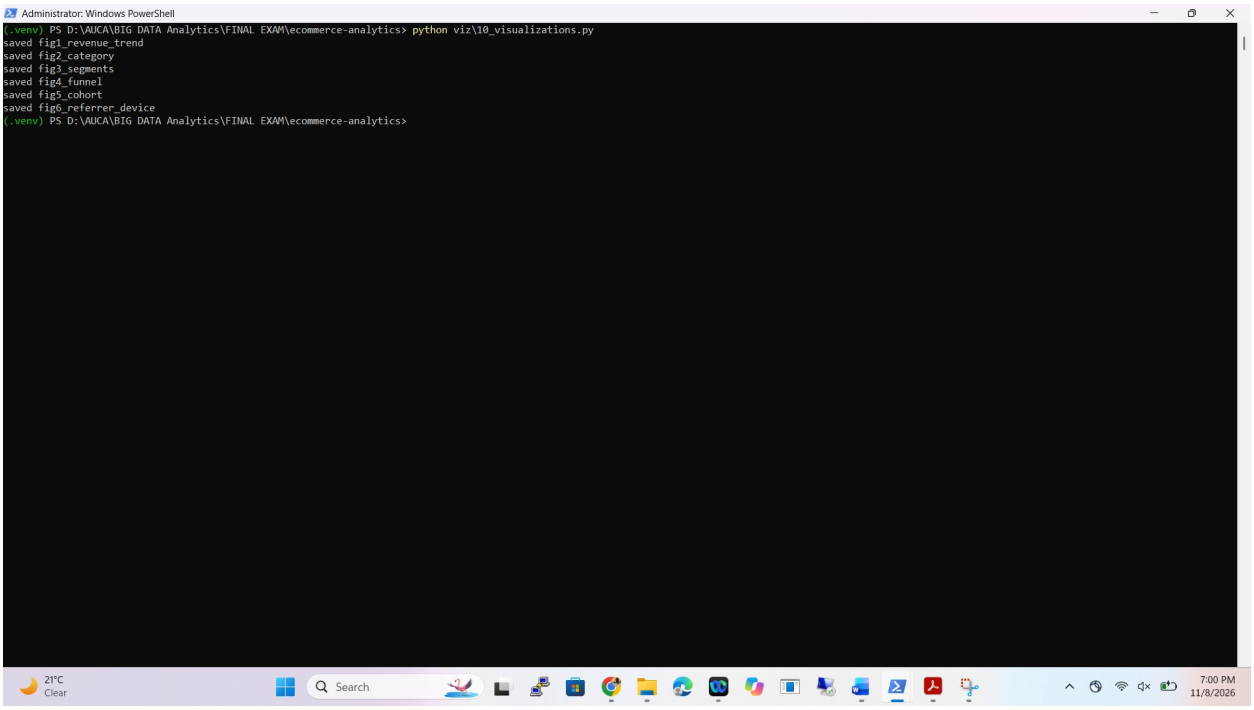
B6 - Spark cleaning and normalisation job with its data-quality report.



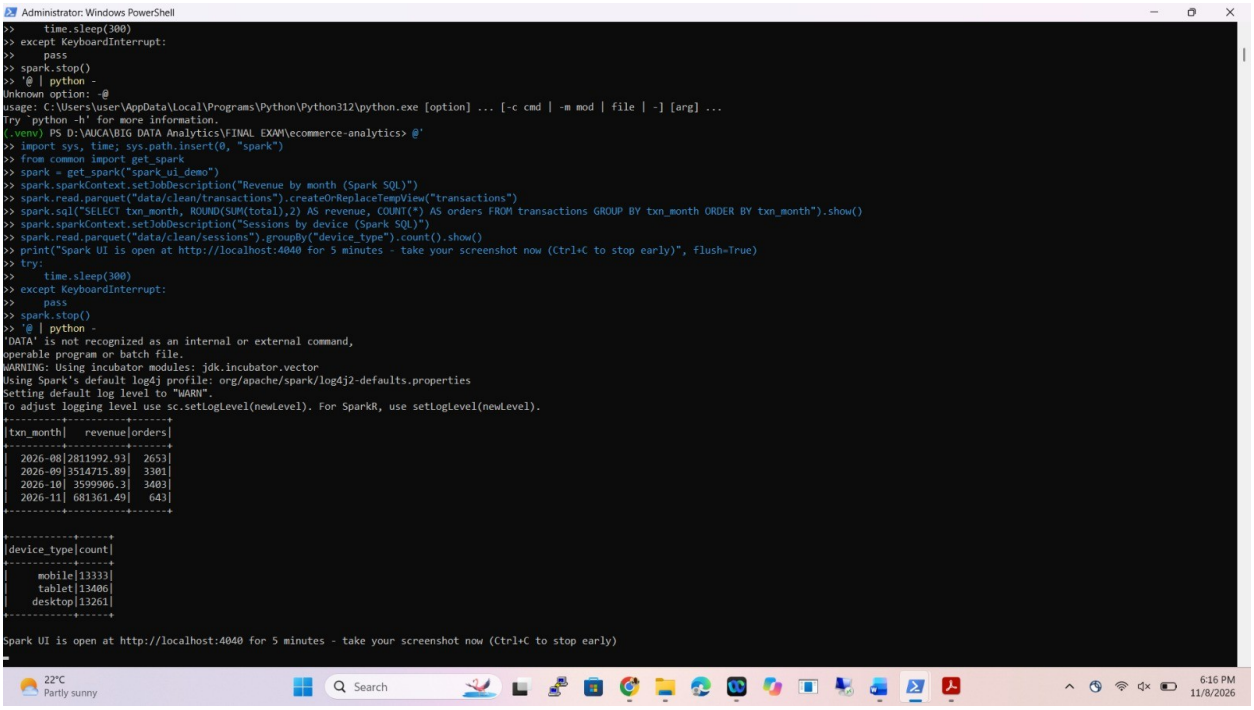
B7 - Spark SQL queries 1-3: top-3 products per category, 7-day moving average, funnel by device.



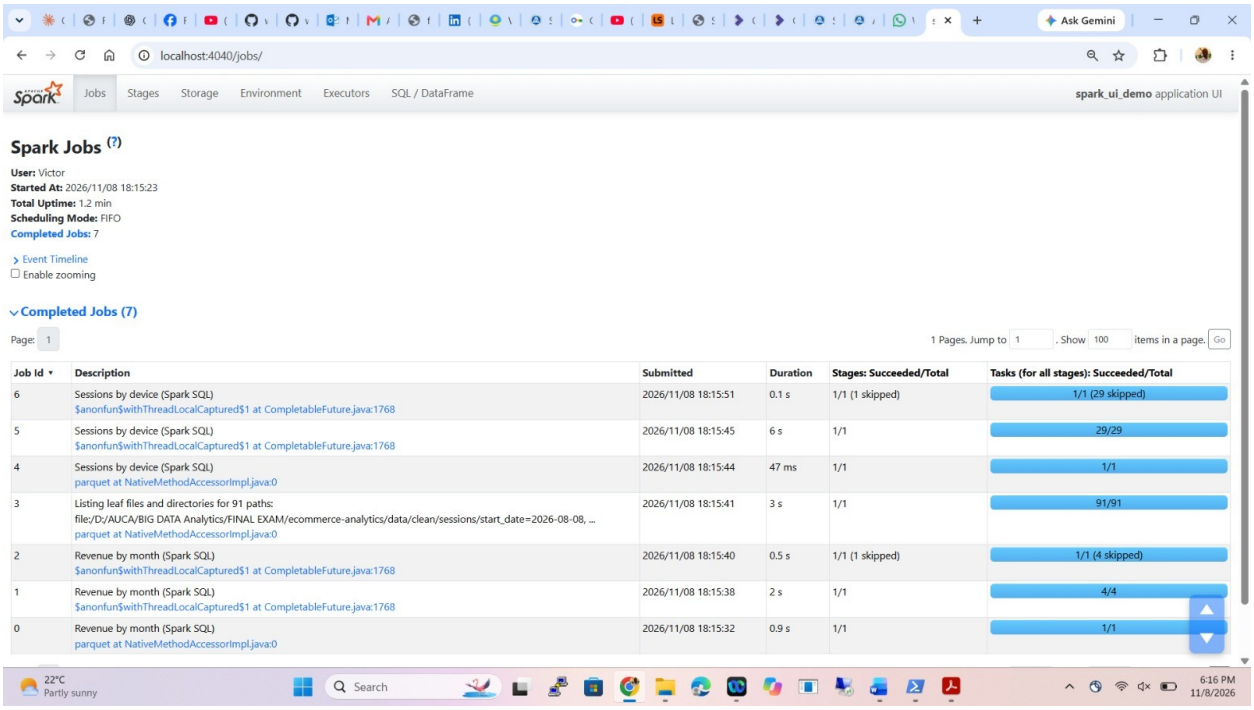
B8 - Integrated query 1: customer lifetime value tiers, correlations and top 10 customers (MongoDB + HBase + Spark). Offline test harness for the two stores.



B9 - Visualisations: viz/10_visualizations.py saving the six figures used in section 5.



B10 - Spark session kept open for the Spark web UI (revenue by month and sessions by device).



B11 - Spark web UI (http://localhost:4040), Jobs page: 7 completed jobs, all stages succeeded.